Отчет по лабараторной работе №2

Простейший вариант

true

Цель работы

Изучить идеологию и применение средств контроля версий.Освоить умения по работе с git.

Задание

Создать базовую конфигурацию для работы с git. Создать ключ SSH. Создать ключ PGP. Настроить подписи git Создать локальный каталог для выполнения заданий по предмету.

Теоретическое введение

Системы контроля версий (Version Control System, VCS) применяются при работе нескольких человек над одним проектом. Обычно основное дерево проекта хранится в локальном или удалённом репозитории, к которому настроен доступ для участников проекта. При внесении изменений в содержание проекта система контроля версий позволяет их фиксировать, совмещать изменения, произведённые разными участниками проекта, производить откат к любой более ранней версии проекта, если это требуется.

В классических системах контроля версий используется централизованная модель, предполагающая наличие единого репозитория для хранения файлов. Выполнение большинства функций по управлению версиями осуществляется специальным сервером. Участник проекта (пользователь) перед началом работы посредством определённых команд получает нужную ему версию файлов. После внесения изменений, пользователь размещает новую версию в хранилище. При этом предыдущие версии

не удаляются из центрального хранилища и к ним можно вернуться в любой момент. Сервер может сохранять не полную версию изменённых файлов, а производить так называемую дельта-компрессию — сохранять только изменения между последовательными версиями, что позволяет уменьшить объём хранимых данных.

Системы контроля версий поддерживают возможность отслеживания и разрешения конфликтов, которые могут возникнуть при работе нескольких человек над одним файлом. Можно объединить (слить) изменения, сделанные разными участниками (автоматически или вручную), вручную выбрать нужную версию, отменить изменения вовсе или заблокировать файлы для изменения. В зависимости от настроек блокировка не позволяет другим пользователям получить рабочую копию или препятствует изменению рабочей копии файла средствами файловой системы ОС, обеспечивая таким образом, привилегированный доступ только одному пользователю, работающему с файлом.

Системы контроля версий также могут обеспечивать дополнительные, более гибкие функциональные возможности. Например, они могут поддерживать работу с несколькими версиями одного файла, сохраняя общую историю изменений до точки ветвления версий и собственные истории изменений каждой ветви. Кроме того, обычно доступна информация о том, кто из участников, когда и какие изменения вносил. Обычно такого рода информация хранится в журнале изменений, доступ к которому можно ограничить.

В отличие от классических, в распределённых системах контроля версий центральный репозиторий не является обязательным.

Среди классических VCS наиболее известны CVS, Subversion, а среди распределённых — Git, Bazaar, Mercurial. Принципы их работы схожи, отличаются они в основном синтаксисом используемых в работе команд.

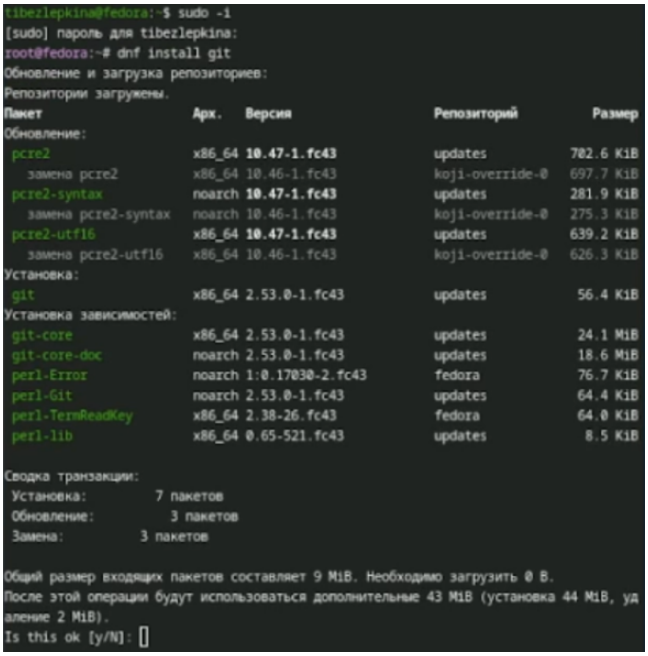
Задание

Создать базовую конфигурацию для работы с git. Создать ключ SSH. Создать ключ PGP. Настроить подписи git. Создать локальный каталог для выполнения заданий по предмету.

Выполнение лабораторной работы

Установка git (рис. fig. 1)

Установка gh (рис. fig. 2)

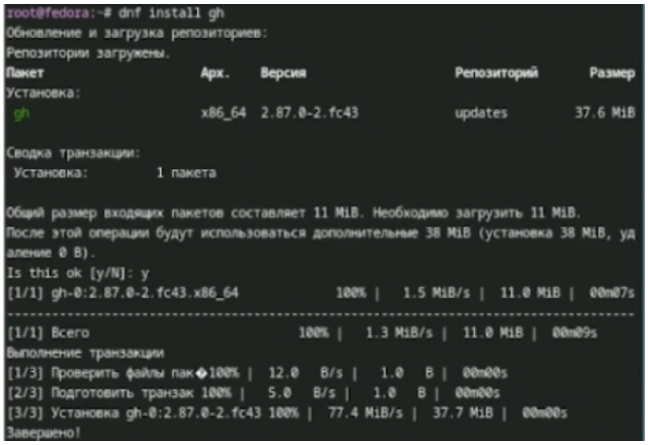


```
tibezlepkina@fedora:~$ sudo -i
[sudo] пароль для tibezlepkina:
root@fedora:~# dnf install git
Обновление и загрузка репозиториев:
Репозитории загружены.
Пакет                    Арх.   Версия                  Репозиторий        Размер
Обновление:
 pcre2                   x86_64 10.47-1.fc43            updates          702.6 KiB
   замена pcre2          x86_64 10.46-1.fc43            koji-override-0  697.7 KiB
 pcre2-syntax            noarch 10.47-1.fc43            updates          281.9 KiB
   замена pcre2-syntax   noarch 10.46-1.fc43            koji-override-0  275.3 KiB
 pcre2-utf16             x86_64 10.47-1.fc43            updates          639.2 KiB
   замена pcre2-utf16    x86_64 10.46-1.fc43            koji-override-0  626.3 KiB
Установка:
 git                     x86_64 2.53.0-1.fc43           updates           56.4 KiB
Установка зависимостей:
 git-core                x86_64 2.53.0-1.fc43           updates           24.1 MiB
 git-core-doc            noarch 2.53.0-1.fc43           updates           18.6 MiB
 perl-Error              noarch 1:0.17030-2.fc43        fedora            76.7 KiB
 perl-Git                noarch 2.53.0-1.fc43           updates           64.4 KiB
 perl-TermReadKey        x86_64 2.38-26.fc43            fedora            64.0 KiB
 perl-lib                x86_64 0.65-521.fc43           updates            8.5 KiB

Сводка транзакции:
 Установка:         7 пакетов
 Обновление:        3 пакетов
 Замена:          3 пакетов

Общий размер входящих пакетов составляет 9 MiB. Необходимо загрузить 0 B.
После этой операции будут использоваться дополнительные 43 MiB (установка 44 MiB, уд
аление 2 MiB).
Is this ok [y/N]:
```

Figure 1: Установка git



```
root@fedora:~# dnf install gh
Обновление и загрузка репозиториев:
Репозитории загружены.
Пакет                    Арх.    Версия                  Репозиторий      Размер
Установка:
 gh                      x86_64  2.87.0-2.fc43           updates        37.6 MiB

Сводка транзакции:
 Установка:         1 пакета

Общий размер входящих пакетов составляет 11 MiB. Необходимо загрузить 11 MiB.
После этой операции будут использоваться дополнительные 38 MiB (установка 38 MiB, уд
аление 0 B).
Is this ok [y/N]: y
[1/1] gh-0:2.87.0-2.fc43.x86_64            100% |   1.5 MiB/s |  11.0 MiB |  00m07s
----------------------------------------------------------------------------------------
[1/1] Всего                                100% |   1.3 MiB/s |  11.0 MiB |  00m09s
Выполнение транзакции
[1/3] Проверить файлы пак►100% |  12.0   B/s |   1.0   B |  00m00s
[2/3] Подготовить транзак 100% |   5.0   B/s |   1.0   B |  00m00s
[3/3] Установка gh-0:2.87.0-2.fc43 100% |  77.4 MiB/s |  37.7 MiB |  00m00s
Завершено!
```

Figure 2: Установка gh

Базовая настройка git,зададим имя и email владельца репозитория (рис. fig. 3)

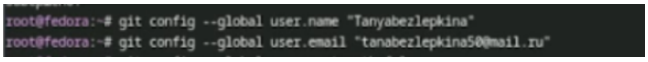


Figure 3: Зададим имя и email владельца репозитория

Настроим utf-8 в выводе сообщений git,зададим имя ветки,а также параметры (рис. fig. 4)

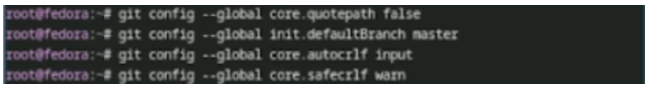


Figure 4: Настроим utf-8 в выводе сообщений git,зададим имя начальной ветки,а также параметры autocrlf и safecrlf

Создадим ssh ключ (рис. fig. 5)

Добавление в github(рис. fig.) Добавим ssh ключ в github

Создадим pgp ключ (рис. fig. 6)

Копируем отпечаток приватного ключа (рис. fig. 7)

Экспортируем ключ по его отпечатку (рис. fig. 8)

Добавим ключ pgp в github(рис. fig. 9)

Настройка автоматических подписей коммитов git (рис. fig. 10)

Авторизация в gh (рис. fig. 11)

Продолжение работы с gh (рис. fig. 12)

Создание репозитория курса на основе шаблона (рис. fig. 13)

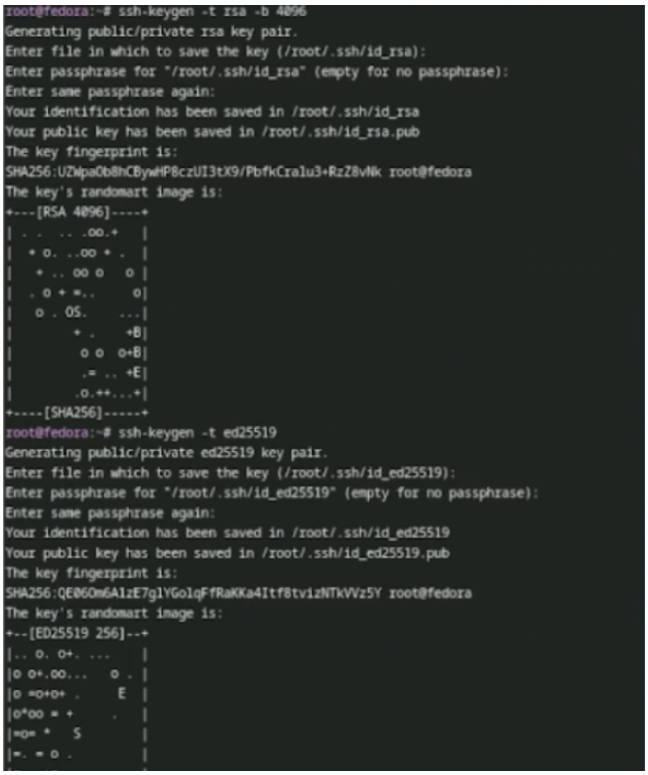
Продолжение загрузки шаблона (рис. fig.) Продолжение загрузки шаблона

Настройка каталога курса (рис. fig. 14)

Отправка файлов на сервер (рис. fig. 15)

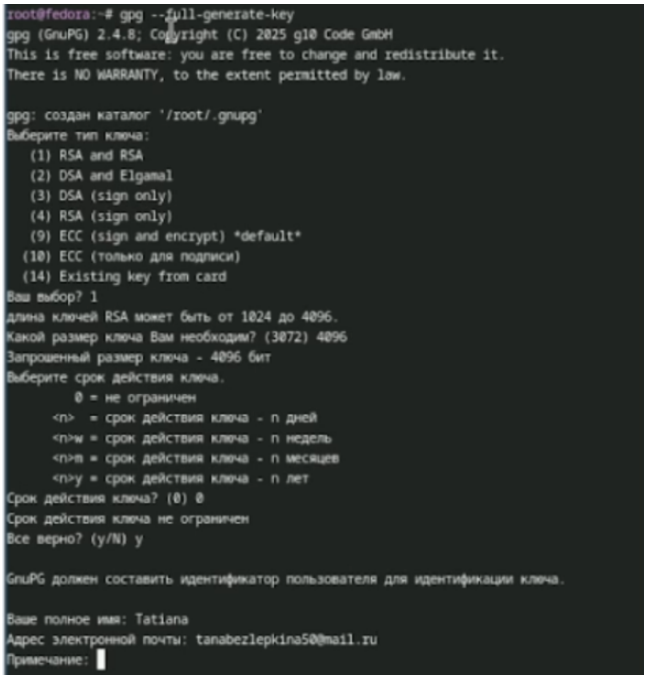
Выводы

Мы приобрели практические навыки работы с github.



```
root@fedora:~# ssh-keygen -t rsa -b 4096
Generating public/private rsa key pair.
Enter file in which to save the key (/root/.ssh/id_rsa):
Enter passphrase for "/root/.ssh/id_rsa" (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /root/.ssh/id_rsa
Your public key has been saved in /root/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:UZWpaOb8hCBywHP8czUI3tX9/PbfkCralu3+RzZ8vNk root@fedora
The key's randomart image is:
+---[RSA 4096]----+
| . .  .. .oo.+   |
|  + o. ..oo + .  |
|   + .. oo o   o |
|  . o + =..     o|
|   o . OS.    ...|
|        + .    +B|
|         o o  o+B|
|         .= .. +E|
|        .o.++...+|
+----[SHA256]-----+
root@fedora:~# ssh-keygen -t ed25519
Generating public/private ed25519 key pair.
Enter file in which to save the key (/root/.ssh/id_ed25519):
Enter passphrase for "/root/.ssh/id_ed25519" (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /root/.ssh/id_ed25519
Your public key has been saved in /root/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:QE06Om6A1zE7glYGolqFfRaKKa4Itf8tvizNTkVVz5Y root@fedora
The key's randomart image is:
+--[ED25519 256]--+
|.. o. o+. ...    |
|o o+.oo...   o . |
|o =o+o+ .     E  |
|o*oo = +     .   |
|=o= *   S        |
|=. = o .         |
```

Figure 5: Создадим ключ ssh



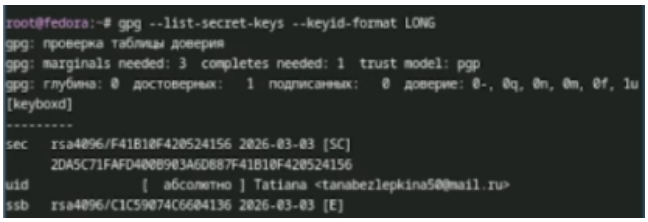
```
root@fedora:~# gpg --full-generate-key
gpg (GnuPG) 2.4.8; Copyright (C) 2025 g10 Code GmbH
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.

gpg: создан каталог '/root/.gnupg'
Выберите тип ключа:
   (1) RSA and RSA
   (2) DSA and Elgamal
   (3) DSA (sign only)
   (4) RSA (sign only)
   (9) ECC (sign and encrypt) *default*
  (10) ECC (только для подписи)
  (14) Existing key from card
Ваш выбор? 1
длина ключей RSA может быть от 1024 до 4096.
Какой размер ключа Вам необходим? (3072) 4096
Запрошенный размер ключа - 4096 бит
Выберите срок действия ключа.
         0 = не ограничен
      <n>  = срок действия ключа - n дней
      <n>w = срок действия ключа - n недель
      <n>m = срок действия ключа - n месяцев
      <n>y = срок действия ключа - n лет
Срок действия ключа? (0) 0
Срок действия ключа не ограничен
Все верно? (y/N) y

GnuPG должен составить идентификатор пользователя для идентификации ключа.

Ваше полное имя: Tatiana
Адрес электронной почты: tanabezlepkina50@mail.ru
Примечание:
```

Figure 6: Генерируем ключ pgp



```
root@fedora:~# gpg --list-secret-keys --keyid-format LONG
gpg: проверка таблицы доверия
gpg: marginals needed: 3  completes needed: 1  trust model: pgp
gpg: глубина: 0  достоверных:   1  подписанных:   0  доверие: 0-, 0q, 0n, 0m, 0f, 1u
[keyboxd]
---------
sec   rsa4096/F41B10F420524156 2026-03-03 [SC]
      2DA5C71FAFD4008903A6D887F41B10F420524156
uid                 [  абсолютно ] Tatiana <tanabezlepkina50@mail.ru>
ssb   rsa4096/C1C59074C6604136 2026-03-03 [E]
```

Figure 7: Выводим список ключей и копируем отпечаток приватного ключа

Figure 8: Экспортируем ключ в формате ASCII по его отпечатку

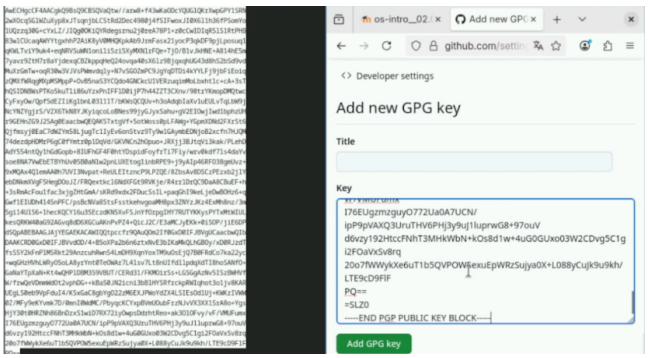


Figure 9: Добавим ключ pgp в github

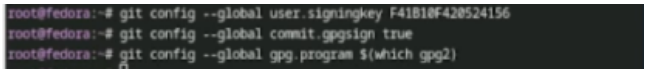


Figure 10: Используя введёный email, укажим Git применять его при подписи коммитов

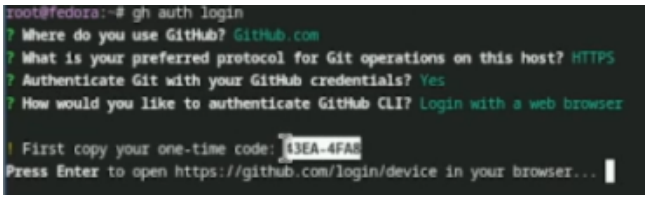


Figure 11: Авторизация в gh

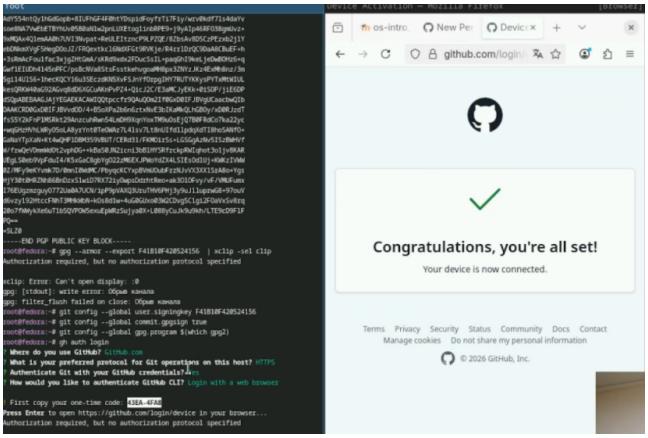


Figure 12: Продолжение работы с gh

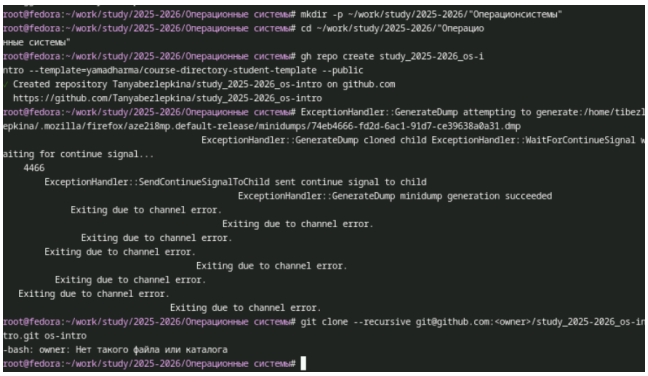


Figure 13: Загрузка шаблона

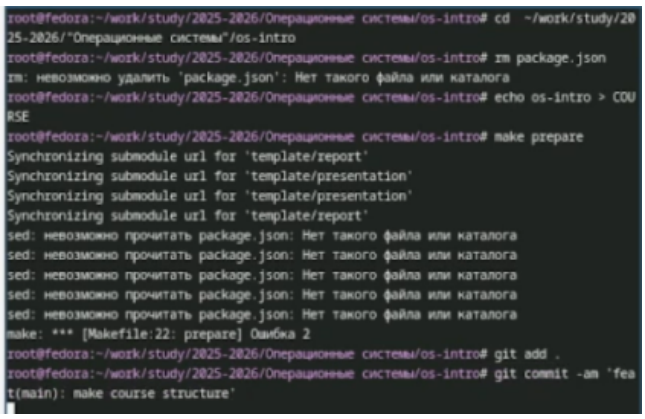


Figure 14: Удаление лишних файлов и создание каталогов

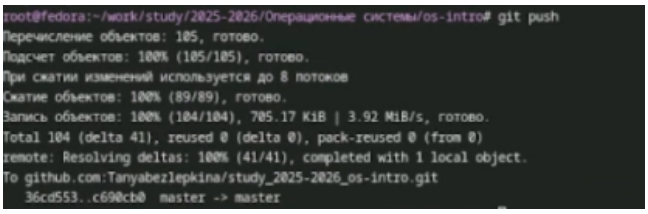


Figure 15: Отправка файлов на сервер

Контрольные вопросы

Вопрос 1: Система контроля версий (VCS) — это инструмент для отслеживания изменений в файлах, который позволяет хранить историю правок, возвращаться к любой предыдущей версии и работать над проектом в команде. Основные задачи: фиксация изменений, возможность отката к старым версиям, параллельная работа нескольких разработчиков, резервное копирование и безопасное проведение экспериментов без риска для основного кода.

Вопрос 2: Хранилище (репозиторий) — это место, где VCS хранит все файлы проекта и полную историю их изменений. Commit (коммит) — это операция создания “снимка” текущего состояния файлов, которая добавляется в историю с комментарием. История — это хронологическая последовательность всех сделанных коммитов. Рабочая копия — это файлы проекта на компьютере разработчика, которые он редактирует в данный момент. Связь: разработчик вносит правки в рабочую копию, затем фиксирует их через коммит, который сохраняется в хранилище и становится частью истории.

Вопрос 3: Централизованные VCS (например, SVN, CVS) имеют единственное хранилище на сервере, и все разработчики подключаются к нему. История хранится только на сервере, без доступа к нему нельзя сделать коммит. Децентрализованные VCS (Git, Mercurial) дают каждому разработчику полную копию репозитория с историей на локальном компьютере. Это позволяет работать офлайн, делать коммиты локально, а сервер используется только для синхронизации. Это быстрее и надежнее.

Вопрос 4: При единоличной работе последовательность действий такая: сначала инициализируешь пустой репозиторий в папке проекта. Затем создаешь или редактируешь файлы в рабочей копии. Когда нужно сохранить результат, добавляешь нужные файлы в индекс и делаешь коммит с описанием. В любой момент можно посмотреть историю и при необходимости откатить проект к любому предыдущему коммиту.

Вопрос 5: Порядок работы с общим хранилищем: сначала клонируешь удаленный репозиторий к себе. Перед началом работы и перед отправкой

своих изменений обязательно загружаешь последние обновления с сервера, чтобы синхронизироваться с коллегами. Затем вносишь изменения в рабочую копию, фиксируешь их локально и отправляешь в общий репозиторий. Если кто-то изменил те же строки кода, возникает конфликт, который нужно разрешить вручную, после чего сделать коммит слияния.

Вопрос 6: Основные задачи, которые решает Git: управление версиями — создание снимков состояния проекта; ветвление и слияние — создание изолированных линий разработки и их объединение; распределенная работа — возможность коммитить офлайн и синхронизироваться позже; отмена изменений — гибкий откат к любой точке истории; организация командной работы через удаленные репозитории.

Вопрос 7: Краткая характеристика основных команд Git: init — создать новый репозиторий; clone — скопировать удаленный репозиторий; status — показать состояние файлов; add — добавить изменения в индекс для коммита; commit — зафиксировать изменения в истории; log — показать историю коммитов; pull — забрать обновления с сервера и слить с текущей веткой; push — отправить локальные коммиты на сервер; branch — посмотреть список веток или создать новую; checkout — переключиться на другую ветку; merge — слить изменения из другой ветки в текущую.

Вопрос 8: Пример работы с локальным репозиторием: инициализируешь репозиторий, создаешь файл, добавляешь его в индекс и делаешь коммит с сообщением. Пример с удаленным: клонируешь удаленный репозиторий, забираешь обновления, вносишь правки, добавляешь их в индекс, коммитишь и отправляешь на сервер.

Вопрос 9: Ветки нужны для параллельной разработки. Они позволяют создавать изолированные линии для новых фич, исправления багов или экспериментов, не трогая стабильный основной код. Когда работа в ветке завершена и проверена, ее можно слить обратно в основную ветку. Это упрощает командную работу и уменьшает риски.

Вопрос 10: Игнорировать файлы при коммите нужно, чтобы не засорять репозиторий мусором и не хранить чувствительные данные. Для этого создается файл .gitignore, в который записывают шаблоны имен файлов и папок, которые Git должен пропускать. Обычно игнорируют временные файлы, результаты компиляции, конфиденциальные данные, огромные папки с зависимостями и персональные настройки IDE.

Список литературы